Technically speaking, the layer's weight is adjusted a bit in the direction opposite to that of the gradient of continuous differentiation of the loss function with respect to the layer's weight. This technique is called *back propagation*. Finally, the process converges at the end of the epoch and we have achieved our desired goal (often accuracy).

Note: Unscrambling a scrambled paper into a clear, flattened sheet through successive steps is the intuitive approach that DL is based on.

Important definitions

Now, let's get familiar with some of the DL related definitions we often encounter.

1. The model consists of a network of layers that have parameters (weights).
2. Layer: *output = function (weight, input)*
3. Tensors: Data structures that represent *input* and *output*
4. Loss function: diff (expected output, actual output)
5. Differentiability: Gradient G = *d (LossFunciont) ⁄ d(Weight)*
6. Gradient descent: Move layer's weight in the opposite direction of G to reduce the loss function on every step.
7. Back propagation: Loopback through optimiser such that the loss function is minimised and runs through the gradient descent.
8. Evaluation: Measures the success (say, the accuracy of a prediction).

Taxonomy and tools

Before learning about the different tools involved in DL – platforms, languages, software – let us first find out more about the different layers in a typical DL deployment scenario.

Table 1 describes the FOSS tool taxonomy, from bottom to top, in the stack.

Note: Keras is the best FOSS way to start with deep learning.

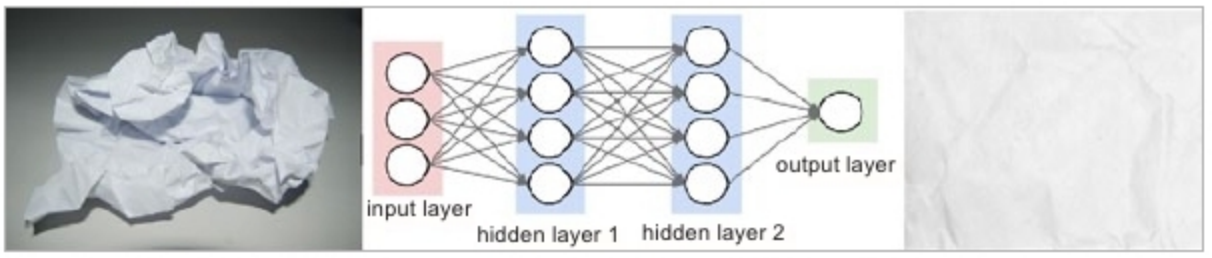

Figure 2: Turning scrambled paper to unscrambled form through a network of nodes

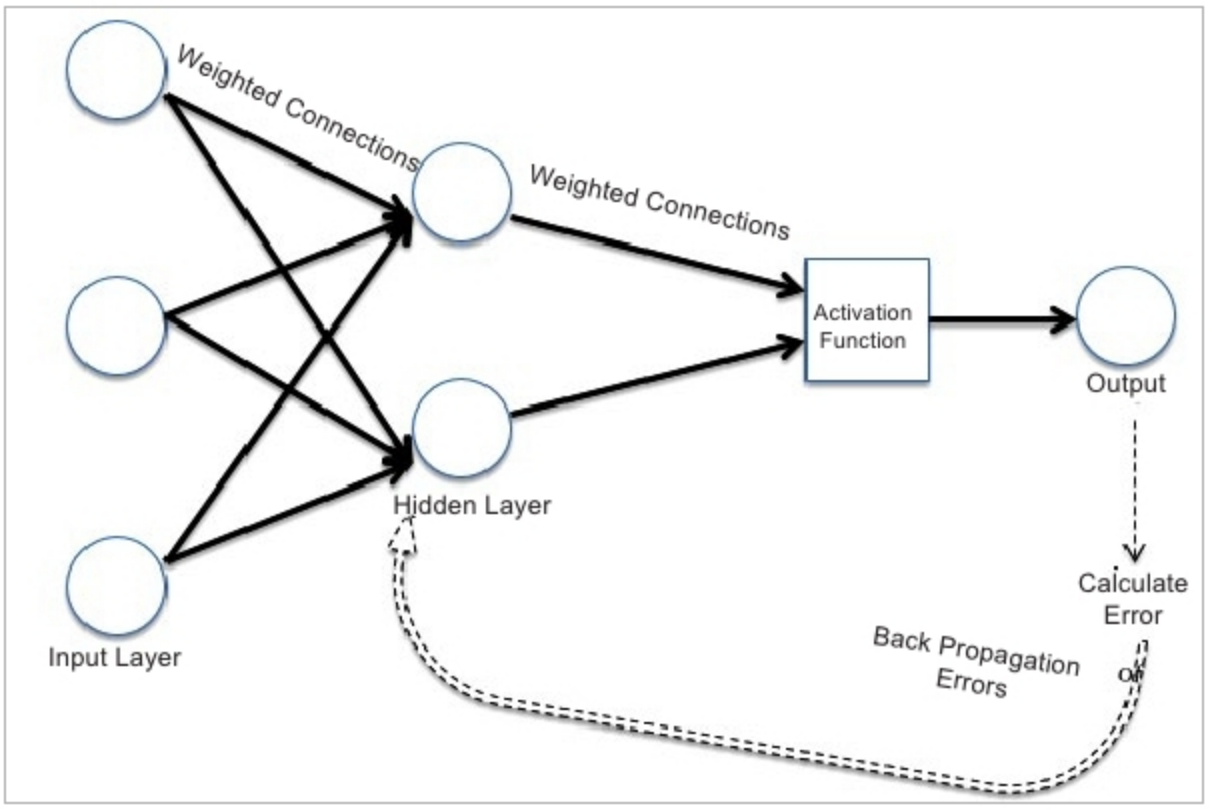

Figure 3: Back propagation illustrated

Keras installation

Keras can usually be deployed in many ways across platforms – Windows, Linux, Mac and even in the cloud (GCP, AWS). For practising DL programming using Keras, it is perfectly fine to have it running in the laptop with conventional CPUs on Windows as well. However, it is possible to book an EC2 GPU instance in AWS and get the benefit of a GPU. That way, one need not procure dedicated GPU hardware and can use the EC2 instance pay-as-you-go model. (However, for longer usage, the cloud may not necessarily be cost effective.)

Here we are going to describe the steps for the installation of Keras and associated dependencies in Ubuntu and Windows 10 using the underlying CPU of the laptop (no GPU needed).

Installation in Ubuntu

1. To install Python-2, use the following code:

```
sudo apt-get update
sudo apt-get install python-pip python
```

2. In case you want to install Python-3 and want to make it the default Python, type:

```
sudo apt-get update
sudo apt-get install python3-pip
python3
alias python=python3
alias pip=pip3
```

3. Install Python-specific scientific dependencies required for DL as follows.
   a. For various Python math packages like NumPy, use:

```
sudo pip install matplotlib
sudo pip install numpy
sudo pip install scipy
sudo pip install PyYAML
```

   b. To view the result (various